Instructions. There are **85 total points**. Answer only inside the provided circles, boxes, figures, or tables, and do not exceed box boundaries. Bubbles must be filled out completely: ● is correct, ✓ ⊙ ⊗ are incorrect. **Permitted materials:** one 8.5x11-inch double-sided note sheet, typed or handwritten. No books, printouts, or other notes, and no electronic devices of any kind (including phones, laptops, tablets, smartwatches, and calculators).

Write your full name here to acknowledge these instructions: **Nick Feamster**

Threat Models and Foundations

1. [4 points]: An AI agent that posts class announcements holds a stored Slack token. An attacker obtains the token and posts "The midterm is canceled" to the course channel. Give the threat model, one item per box: the **asset** harmed (not the token itself), the **property** violated, a plausible **adversary** and their motive, and the **capability** the attacker needed.

Asset:	the channel; the instructor's voice	Property:	integrity (accept authenticity)
Adversary:	a prankster or a student wanting a delay	Capability:	the stored token: acts as the instructor, no second factor

2. [3 points]: Every security system has a *root of trust*: the point where the chain of trust stops and you simply accept something. For each, name the key or certificate that is accepted without being checked, and who ships it.

A browser's TLS connection to a web site:	the root CAs shipped with the OS or browser (accept the vendor that ships the root store)
DNSSEC validation of a DNS answer:	the root zone's public key built into the resolver software
RPKI's check that an AS may announce a prefix:	the regional registries' trust-anchor keys in the validator (accept IANA/ICANN as the top of the delegation)

3. [3 points]: A payments startup discovers that a production API key committed to a public GitHub repository eight months ago has been used by an outsider to issue refunds. Among the failure modes in Anderson's "Why Cryptosystems Fail," this is primarily a failure of: (Select the best answer.)

◯ The cryptographic algorithm protecting the key

● Key management and operational procedure

◯ A remote software vulnerability in the API server

◯ A fake terminal or skimmer

◯ Usability of the customer-facing product

Initials: ____

4. [4 points]: Case study. In 2012 an anonymous researcher logged into hundreds of thousands of home routers that still had their factory default passwords, used them to scan the entire Internet, and published the data set (the "Carna botnet"). The owners were never asked. Today a graduate student proposes to use the published data set to study how address usage has changed since 2012. Using *respect for persons*, give one argument **against** the original collection; using *beneficence*, give one argument **for** the reuse today.

Against the collection (respect for persons):

The owners' devices were accessed and modified without informed consent: never asked, never told, no way to opt out, so they were treated as means rather than persons.

For the reuse (beneficence):

The data already exists and cannot be un-collected; reuse adds no new risk to the owners (decade-old address-level data) and the benefit, a baseline for Internet growth, is real; risk against benefit favors reuse even though it did not favor collection.

Key Management and Public Key Infrastructure

5. [4 points]: Which of the following statements are true? (Select all that apply.)

- ● A browser uses asymmetric cryptography to establish a symmetric key, which then encrypts the traffic
- ● To send Bob a message only he can read, Alice encrypts it with Bob's public key
- ○ To sign a message so others can verify it came from her, Alice uses Bob's public key
- ● A certificate is a signed statement binding an identity to a public key, with an issuer and validity period
- ○ Integrity means that only authorized parties can read a message

6. [4 points]: The first time you connect to a new server with SSH, the client shows a key fingerprint and asks whether to trust it. Explain how Mallory, on the network path, mounts a man-in-the-middle attack on that first connection, and what a careful user must do to defeat it.

(Answer inside the box)

Solution: The client has no prior key, so Mallory intercepts the connection, presents her own key and fingerprint, and connects onward to the real server as the client; she shares one session key with each side and reads and re-encrypts everything. Defeat: confirm the fingerprint out of band over a channel Mallory does not control (the admin's web page, a phone call, a key published in advance); afterwards known-hosts pins the key.

7. [4 points]: In Assignment 1 you captured your own HTTPS traffic with Wireshark and served your site with a self-signed certificate.

(a) Even though the payload was encrypted, name **two** things an on-path eavesdropper could still learn from an HTTPS connection like the one you captured.

Visible: SNI in the ClientHello; the server's and client's IP addresses; sizes and timing of records; the preceding DNS query. Any two.

(b) The browser warned about your self-signed certificate until you took one step. What was it, and what could someone who performed that same step on your laptop without your knowledge then do to your *other* HTTPS traffic?

Add the certificate to the machine's (or browser's) trust store, making it a root of trust. Anyone who can install a root on your machine can mint a certificate for any site and sit on the path as a man in the middle with no browser warning.

8. [3 points]: Alice posts her public key in the course Slack channel so people can send her encrypted mail. This solves the key-distribution problem for anyone who wants to write to her. ◯ Yes ● No Explain why or why not.

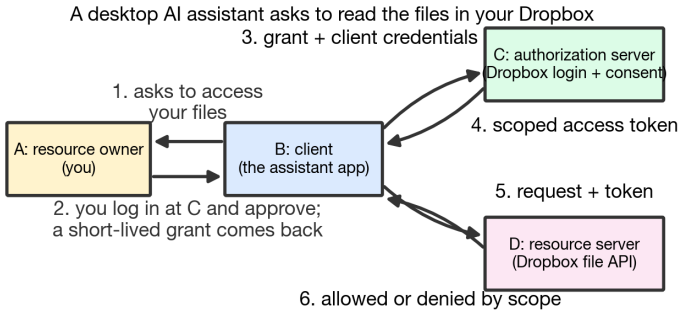
(Answer inside the box)

Solution: No. Anyone who can post as Alice, alter the message, or sit between the reader and the channel can substitute their own key, and the reader cannot tell. Distribution needs a trusted binding of identity to key (a certificate from a party already trusted, a fingerprint confirmed out of band, a pinned key), not just a place to post it. Partial credit for Yes: posting a public key does remove the n^2 shared-key part of the problem (one key pair each instead of a shared key per pair); what it does not solve is binding the key to Alice.

Authentication and Access Control

9. [4 points]: A desktop AI assistant asks to read the files in your Dropbox. The diagram shows the OAuth flow between four parties, A through D. OAuth names four roles: **resource owner**, **client**, **authorization server**, and **resource server**. Write the letter of the party that plays each role.

Resource owner:	A
Client:	B
Authorization server:	C
Resource server:	D



10. [3 points]: The agent that posts the class agenda presents a token to Slack, which checks the token and then lets it post in one channel but not delete messages. Which statements use the right term? (Select all that apply.)

● Presenting the token and having Slack verify it is *authentication*

◯ Presenting the token is *authorization*; checking it is *identification*

● Allowing posting but not deletion is *authorization*, enforced by access control

◯ Allowing posting but not deletion is *authentication*

● Recording that the agent posted at 3:05 pm is *logging*, which supports accountability

Grading note: the first option is ambiguous (the token is an authorization credential, so checking it can be read as authorization; read with commas, it is authentication). It is credited whether marked or blank.

11. [3 points]: A budgeting app offers to track your spending if you give it your username and password for your bank's web site.

(a) Explain why this violates least privilege.

(Answer inside the box)

Solution: With your password the app can do anything you can (read every account, move money, change settings) and looks like you in every log; it cannot be limited, audited, or revoked short of a password change. Least privilege: grant only what the task needs.

(b) Describe the narrow OAuth scope the bank should grant instead, as **who** (subject) may do **what** (verb) to **which data** (object).

Subject:	Verb:	Object:
the budgeting app (accept "the app acting for the user")	read (only); accept "track" or "view"	transactions and balances on the accounts the user selects

Initials: ____

Denial of Service

12. [4 points]: To stop a denial-of-service attack coming from one region, campus IT installs a firewall at the edge of the network that inspects every incoming connection and drops the attack traffic. Deploying the firewall has introduced a **new** denial-of-service vulnerability. ● Yes ◯ No Explain why or why not.

(Answer inside the box)

Solution: Yes. The firewall has finite memory and CPU, and every packet, including all the attack traffic, must now pass through it and be inspected. Enough traffic exhausts its state table or CPU, and when it fails nobody gets through, so the appliance added to stop the attack is the new, easier target. Any device placed in front of a service becomes the next finite resource to exhaust.

13. [4 points]: A new chat service allocates a 64 KB message buffer for every connection request the moment it arrives, before the client has sent anything else. Describe one way an attacker exploits this, and redesign the protocol in the spirit of SYN cookies so that the attack no longer works. A sentence or two of design is enough; no cryptographic detail is needed.

(Answer inside the box)

Solution: Exploit: flood the server with connection requests from spoofed or throwaway sources and never follow up; each costs the attacker one small packet and the server 64 KB, so memory runs out first. Redesign: do no work on the first packet; reply with a small cookie derived from the client's address and a secret, and allocate the buffer only when the client returns it, proving it is real and reachable. The client spends effort before the server does.

Routing Security

14. [2 points]: YouTube announces its addresses as a /22. Pakistan Telecom, intending to block YouTube inside Pakistan, announces two /24s covering the same space, and the announcement leaks to the whole Internet. Why did the world's YouTube traffic go to Pakistan? (Select the best answer.)

◯ Pakistan Telecom's AS path was shorter

● Routers prefer the more specific (longer) prefix, so the /24s won over the /22

◯ YouTube's RPKI certificate had expired

◯ Pakistan Telecom signed its announcement with BGPsec

15. [3 points]: Name **two** different things an attacker can do with someone else's addresses or traffic once a hijacked announcement is believed.

(Answer inside the box)

Solution: Eavesdrop on the traffic; black-hole it (denial of service, as Pakistan did); forward it to the real destination as a man in the middle (needs a path to the destination); or use hijacked, unused space as a disposable source for spam, then withdraw the route to defeat address-based block lists. Any two.

16. [4 points]: In BGPsec, each AS signs an attestation covering the AS path so far **and the AS it is sending the announcement to**. AS 160 originates a prefix and announces it to AS 174, which announces it to AS 15169. Suppose each AS signed the path so far but **not** the AS it was sending the announcement to. What could a dishonest AS 15169 announce to its own neighbors that would still pass validation, and why does 160's signature naming 174 prevent it?

(Answer inside the box)

Solution: Signing only one's own hop lets AS 15169 drop 174 from the announcement it received and re-announce 160 then 15169 with the origin's genuine signature intact: a shorter path with a correct origin, so origin validation passes. With the next hop signed, 160's attestation says "to 174"; a path omitting 174 would need 160 to have signed "to 15169", which it never did and 15169 cannot forge.

Web Security

17. [3 points]: A page is loaded from `https://www.cs.uchicago.edu/courses`. Which of the following share its **origin** under the same-origin policy? (Select all that apply.)

- ● `https://www.cs.uchicago.edu/people`
- ◯ `http://www.cs.uchicago.edu/courses`
- ◯ `https://cs.uchicago.edu/courses`
- ● `https://www.cs.uchicago.edu/courses?term=autumn`
- ● A script the page loads from `https://cdn.example.com/jquery.js` runs in the page's origin

18. [3 points]: A web application runs the pseudocode

```
run_shell("ls " + request.parameter("dir"))
```

when a user requests a directory listing. What class of vulnerability is this, and what would an attacker put in the `dir` parameter to make the server run a command of the attacker's choosing? (An English description of the value is fine.)

(Answer inside the box)

Solution: Shell (command) injection: request data is pasted into a command and the shell cannot tell data from code. A dir value such as "; <command>" (any separator such as a semicolon or a pipe, followed by the attacker's command: a reverse shell, reading a password file, deleting files) runs ls and then that command with the web application's permissions. Fix: never build commands from strings; allow-list the value or pass it as an argument, not through a shell.

19. [3 points]: A discussion forum displays each comment exactly as the author typed it, and a user posts a comment containing a `<script>` tag. Is this ◯ reflected or ● stored cross-site scripting? Why does the same-origin policy not protect the forum's other users?

(Answer inside the box)

Solution: Stored. The same-origin policy does not help because the forum itself delivers the script, so it runs in the forum's origin with full access to its DOM, cookies, and storage; the policy separates origins, but the attacker's code is inside the trusted one.

20. [3 points]: You are logged in to your bank in one tab and visit a page showing a picture of a muffin in another.
(a) What kind of attack can the muffin page mount against your bank account, and why does it work?

(Answer inside the box)

Solution: Cross-site request forgery: the muffin page embeds a hidden image or request whose URL is a bank action, and the browser attaches your bank session cookie because it sends a domain's cookies with every request to that domain, whatever page caused it.

(b) Name one defense that stops it: a per-session token in the bank's own forms (unreadable to the muffin page under the same-origin policy), or SameSite cookies, now the default, which are withheld on cross-site subresource requests and POSTs

Initials: ____

Web Privacy

21. [3 points]: `nytimes.com` and `walmart.com` both embed a script from `tracker.example`. Explain how `tracker.example` learns that the same person visited both sites.

(Answer inside the box)

Solution: On the first visit the script sets a tracker.example cookie holding an identifier. On every later page embedding the script, the browser (1) sends that cookie back automatically, because a domain's cookies go with every request to that domain whatever page caused it, and (2) sends a Referer header naming the page. The tracker sees the same identifier arrive from nytimes.com and then walmart.com. Accept any two of: fetching embedded third-party resources automatically, sending the tracker's cookie with every request to its domain, sending Referer (or the script reporting the page URL).

22. [3 points]: You are not signed in to any account, you have deleted all your cookies, and you use a private-browsing window. A tracking script embedded on the sites you visit can still recognize that your visits come from the same browser. ● Yes ○ No Explain why or why not.

(Answer inside the box)

Solution: Yes. The script can fingerprint the browser with no stored state: user-agent, fonts, plugins, screen size, time zone, canvas rendering combine into a value that is unique or nearly so (one in hundreds of thousands in the class demo). It is recomputed on every visit, so deleting cookies and staying signed out does not reset it; the tracker can even re-create the cookie.

DNS Security and Privacy

23. [3 points]: Which of the following statements are true? (Select all that apply.)

○ DNSSEC provides confidentiality: an eavesdropper cannot see which names you look up

● Encryption provides confidentiality; signatures provide integrity

○ A browser cannot resolve names itself; every lookup must go through the operating system's stub resolver

● With Oblivious DNS, no single server sees both your address and your query

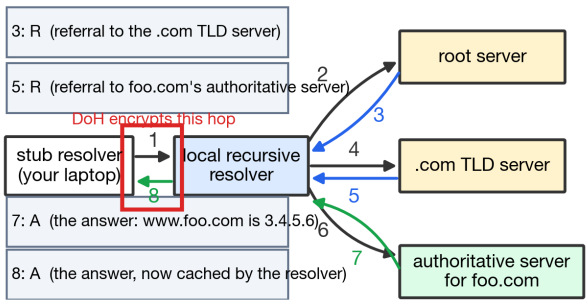
● DNS-over-HTTPS moves trust from your network operator to the resolver provider rather than eliminating it

24. [3 points]:

The diagram shows a lookup of `www.foo.com` with nothing cached.

(a) In each dashed box, label message 3, 5, 7, or 8 with **R** if it is a referral to another server or **A** if it is an answer carrying the record.

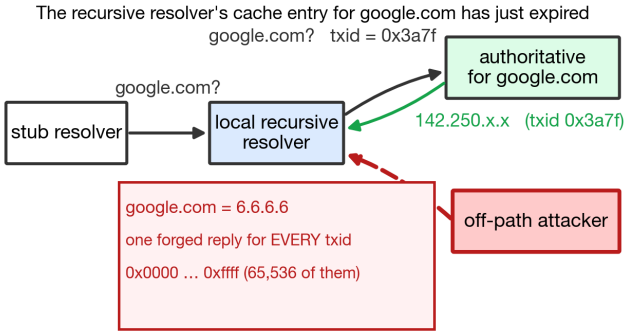
(b) Draw a box around the link (both directions) that DNS-over-HTTPS encrypts.



Initials: ____

25. [3 points]:

The resolver's cache entry for `google.com` has expired; it re-queries the authoritative server with transaction ID `0x3a7f`, which the **off-path** attacker cannot see. In the dashed box, write what the attacker sends (its content, and how many); below, explain in one sentence why it works.



Solution: Forged answers (google.com = 6.6.6.6) with every 16-bit ID, all 65,536 (accept "every possible ID"), spoofed from the authoritative server's address. The ID is the resolver's check, and the attacker can enumerate every ID without seeing the query; one that matches and beats the real reply is cached. Credit answers that add source spoofing or the race.

(Answer inside the box)

Course Feedback

26. [1 point]: Interest (1=Boring!; 10=Amazing!): 8 Difficulty (1=Too easy; 10=Too hard): 6

27. [1 point]: (a) One thing you liked about the course so far. (b) One suggestion for improvement:

(Answer inside the box)

Solution: The debates. More time on the hands-on activities.